Cryptography

University of Insubria

Secure Encryption

When can we say that a cipher is "secure"?

- a cipher is secure if the ciphertext "appears random," that is, the ciphertext does not leak any information about the plaintext

Given c, c' should be difficult to decide if $m \neq m'$

Pseudo-random permutations

How do we build a secure secret key encryption scheme?

- a secure scheme should be like a pseudo-random permutation over the alphabet

A permutation (over a finite set S) is pseudo-random (PRP) if a poly time adv cannot distinguish it from a permutation picked at random from the set of all possible permutations over S

Pseudo-random permutations

Pseudo-random permutation $F_k : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^n$ s.t.

- **Efficient**: F_k is computable (and invertible) in polynomial time
- **Secure**: F_k with a random key cannot be distinguished by a poly time adv from a random function from the set of all functions from $\{0,1\}^n$ to $\{0,1\}^n$

How many permutations $\{0,1\}^n \rightarrow \{0,1\}^n$? $2^n!$

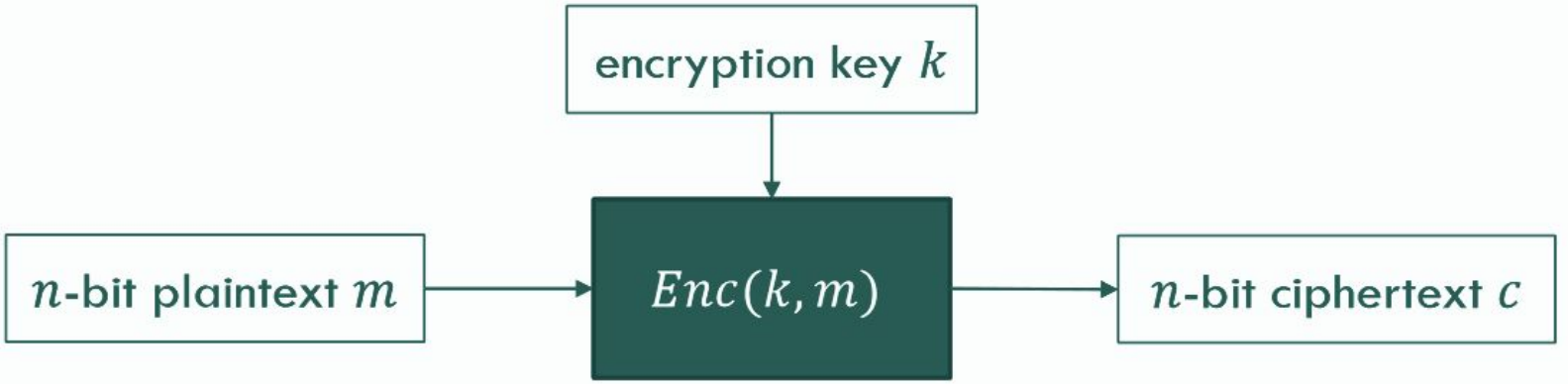
Pseudo-random Generator

A pseudorandom number generator (PRNG) is a deterministic algorithm that generates numbers that "look" random

- the generator takes a "small" number (seed)
- from this small number it produces a very long stream of bits

Block Ciphers

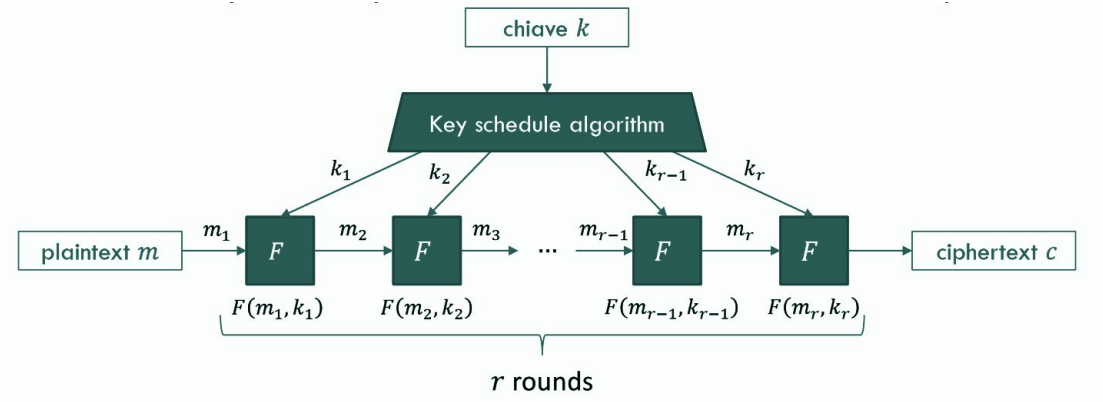
A block cipher is an algorithm that allows the encryption of blocks of fixed length. The length of the blocks is called "blocksize"



Block Ciphers

A block cipher is a (keyed) permutation on the possible blocks of n bits

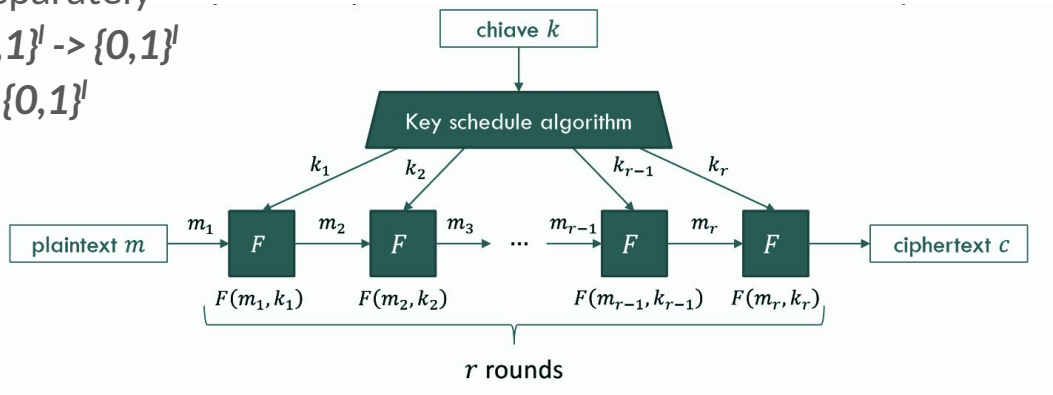
- It is a reversible function that uniquely associates each block with another block. The permutation is determined by the chosen key



Block Ciphers

Building blocks

- The plaintext is divided in bit blocks of equal length l (typical block lengths is 128, or even 256)
- Each block is encrypted separately
- Block cipher: $F : \{0,1\}^n \times \{0,1\}^l \rightarrow \{0,1\}^l$
- For every k, F_k is a PRP on $\{0,1\}^l$



Block Ciphers

How to represent a PRP in a concise way?

- Construct a random looking permutation ***F*** over a large block with many "small" random looking functions f_i

Confusion-Diffusion Principle

Properties of a secure cipher identified by Shannon

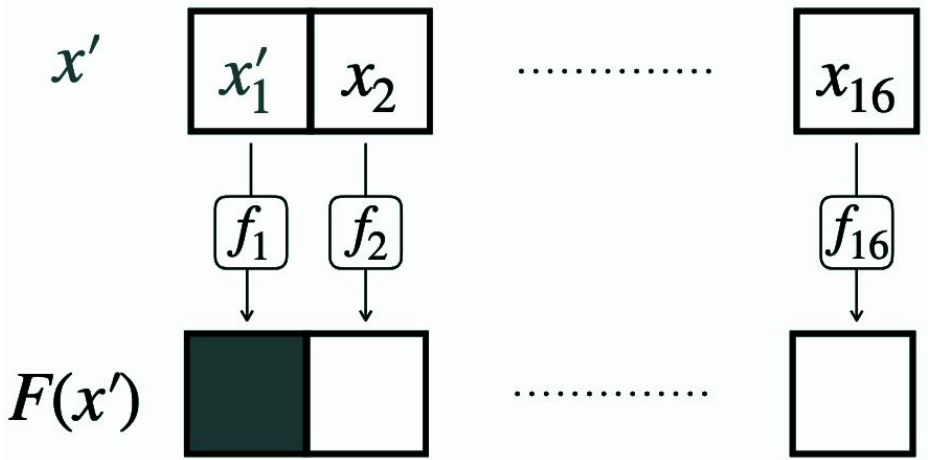
- Confusion
  - obscuring the local correlation between the input and output by varying the application of the key to the data
- Diffusion
  - hiding the plaintext statistics by spreading it over a larger area of ciphertext

Repeat(Round = confusion step + diffusion step)

Confusion-Diffusion Principle

Confusion

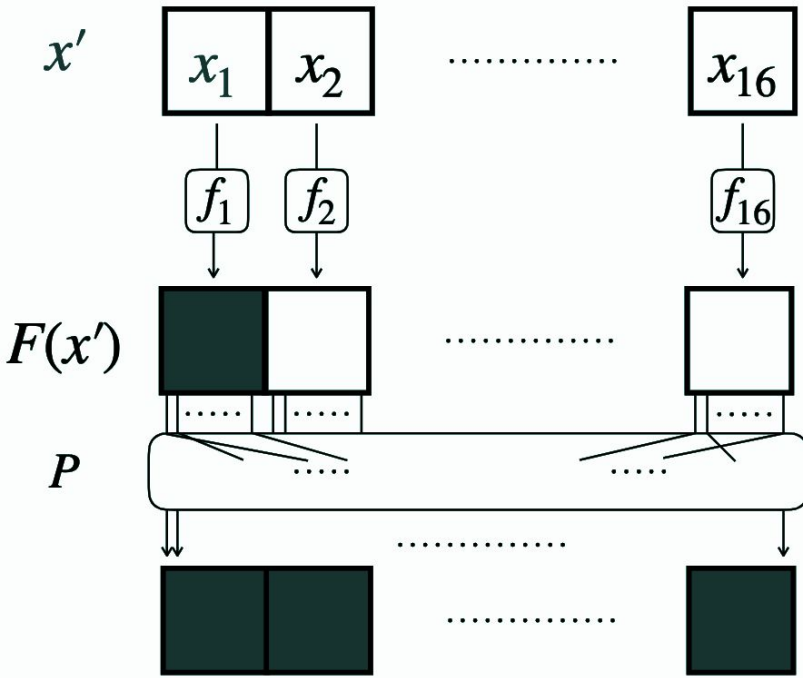
- $F_k(x) = f_1(x_1) \,||\, f_2(x_2) \,||\, .. \,||\, f_{16}(x_{16})$
- f_i are called round functions



Confusion-Diffusion Principle

Diffusion

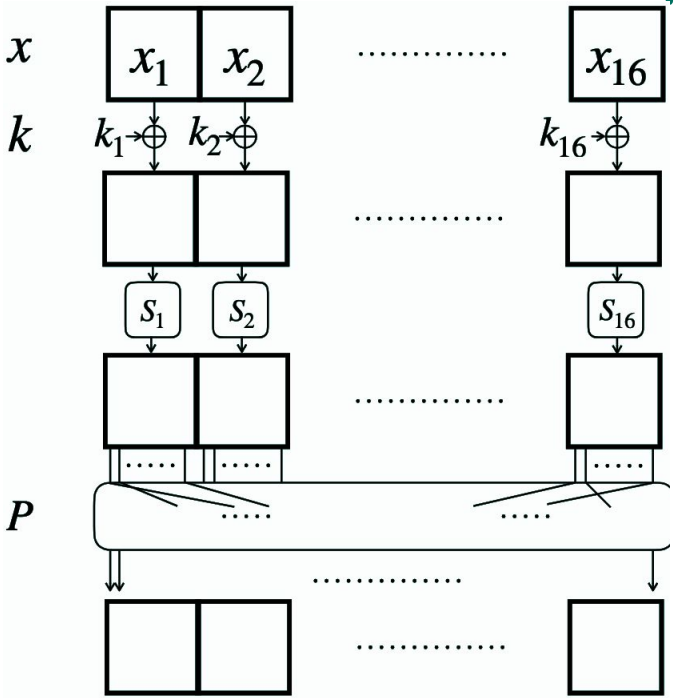
- Add a mixing permutation P
- Spread the local change all over the entire block



Subsitution-Permutation Networks

Instead of using functions fi defined by the key

- fixed substitutions (S-boxes)
- the key is mixed with the plaintext using a XOR
- a subkey derived from the original key at every round



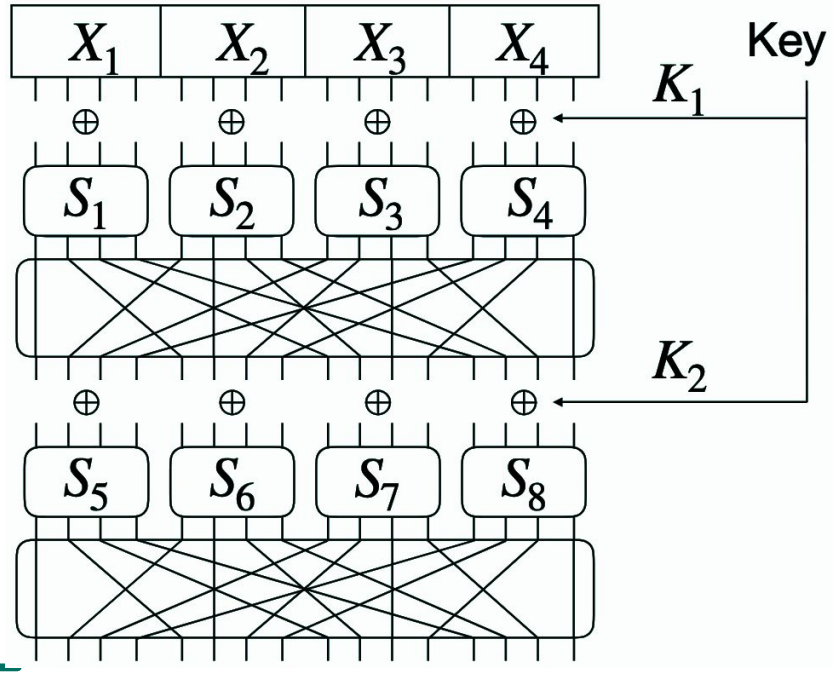
Subsitution-Permutation Networks

One round is formed by

- Key mixing $x \leftarrow x \oplus k_i$
- Substitution: $x \leftarrow S_1(x_1) \,||\, .. \,||\, S_4(x_4)$
- Permutation: permute the bits (fixed P)

//

- S-boxes and permutation are public
- Only secret is the master key



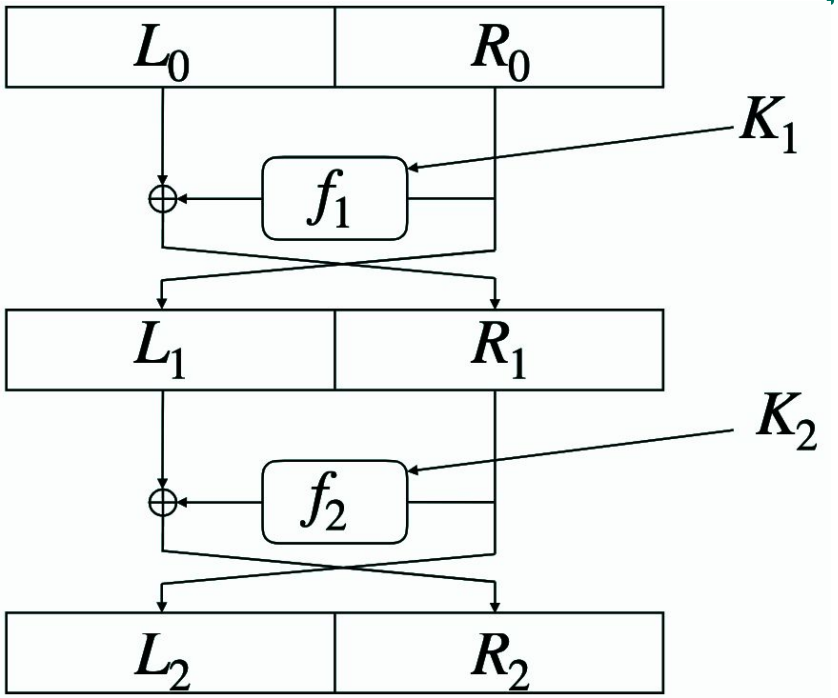
Feistel Networks

One round of a Feisten network is defined as

- Round(L, R) = (R, L ⊕ f(R))

If f_i are pseudo-random functions with independent keys then a 3-round Feistel network that uses is a PRP

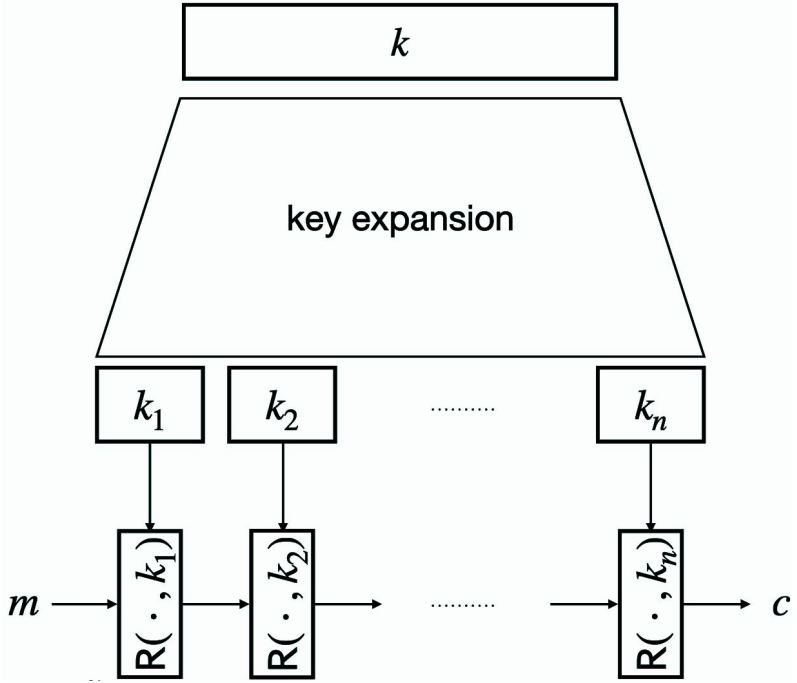
// in real-world much more than 3 rounds are used



Block Cipher

R(., k) is a round function

- DES
  - n=16 round
- 3DES
  - n=48 round
- AES-128
  - n=10, 12, 14 round



DES - Data Encryption Standard

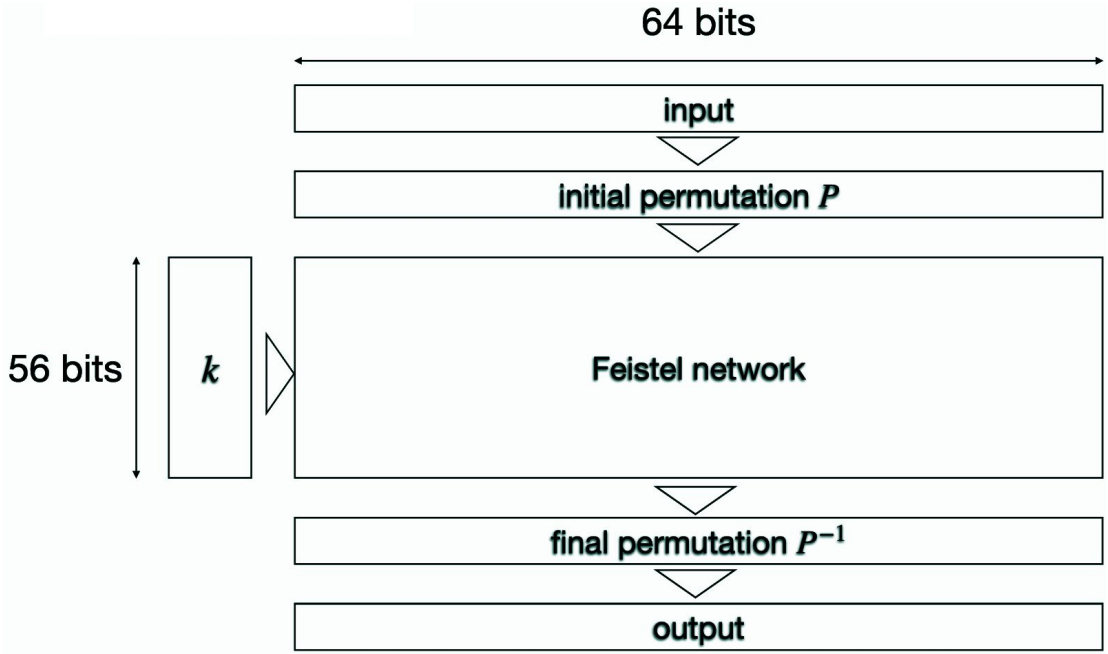
Encryption schema

- Proposed by IBM in 1975, standardized in 1977
- DES is a 16-round Feistel network
- block size = 64 bits, key size = 56 bits

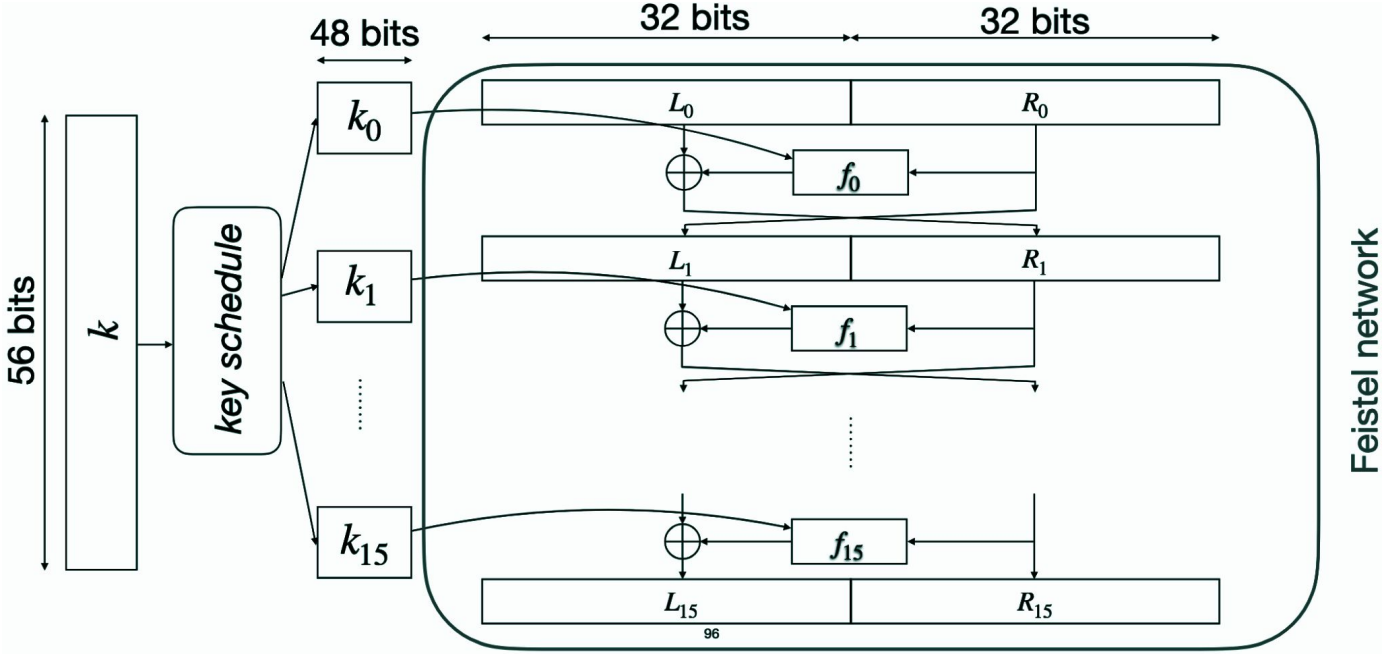
The key

- is (random) 64-bits string
- the 8th, 16th, 24th,…,64th bits are discarded
- the size is 56 bits

DES - Data Encryption Standard



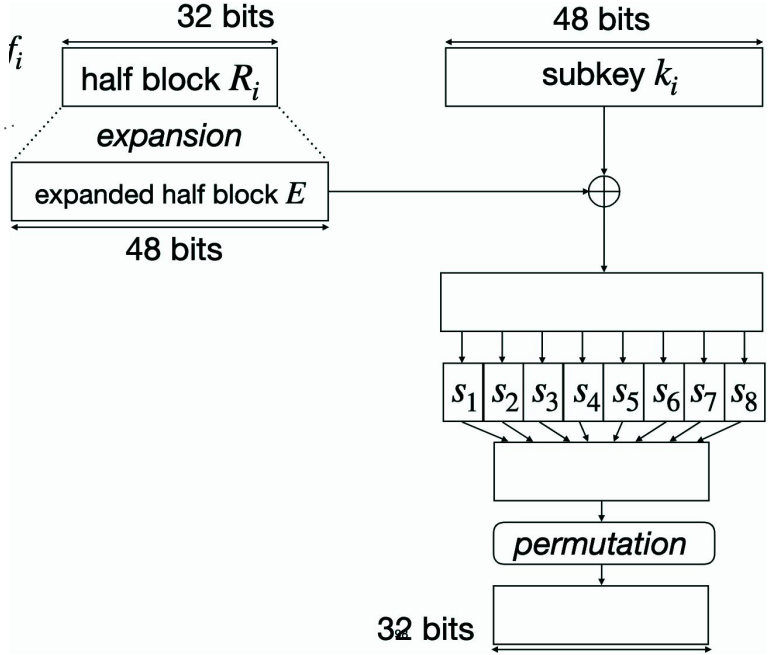
DES - Data Encryption Standard



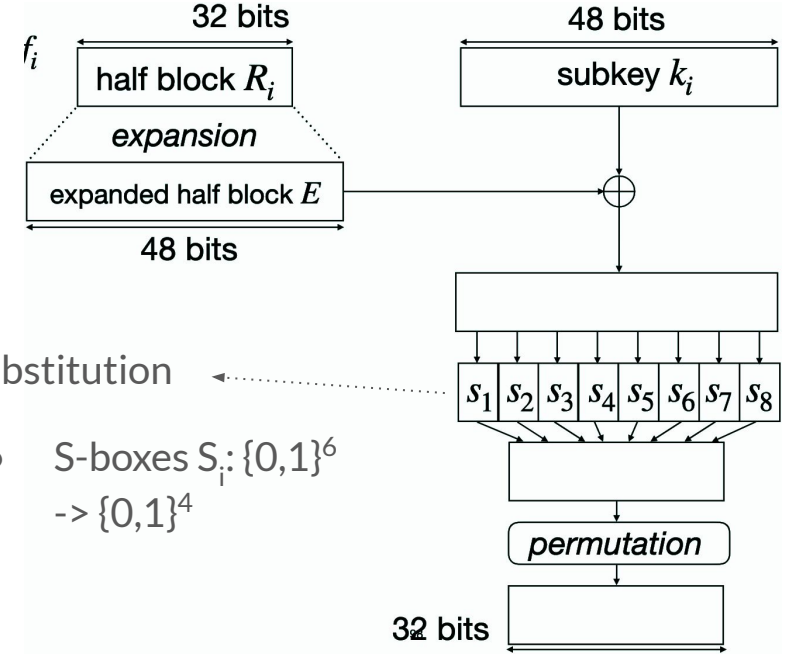
DES - Data Encryption Standard

Expansion:

- bits 1, 8, 9 ,12 , ..
  32 are duplicated



DES - Data Encryption Standard



Substitution

- S-boxes S_i: $\{0,1\}^6$ -> $\{0,1\}^4$

S-box

4 middle input bits

2 outer input bits																
	S_5	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
	00	0010	1100	0100	0001	0111	1010	1011	0110	1000	0101	0011	1111	1101	0000	1110	1001
	01	1110	1011	0010	1100	0100	0111	1101	0001	0101	0000	1111	1010	0011	1001	1000	0110
	10	0100	0010	0001	1011	1010	1101	0111	1000	1111	1001	1100	0101	0110	0011	0000	1110
	11	1011	1000	1100	0111	0001	1110	0010	1101	0110	1111	0000	1001	0101	0100	0101	0011

DES - Data Encryption Standard

DES has 4 weak keys and 12 semi-weak key pairs that results in a highly insecure encryption process

Weak keys

- 0x0101010101010101
- 0xFEFEFEFEFEFEFEFE
- 0xE0E0E0E0F1F1F1F1
- 0xE1E1E1E1F0F0F0F0

If wk is a weak key, then $DES_{wk}(DES_{wk}(m)) = m$

DES - Data Encryption Standard

Security Problems

- Small Key Size (56-bit Key) - Brute force
- Weak and Semi-Weak Keys

DES variants

- 3DES
  - $3DES((k1, k2, k3), m) = DES(k1, DES^{-1}(k2, DES(k3, m)))$

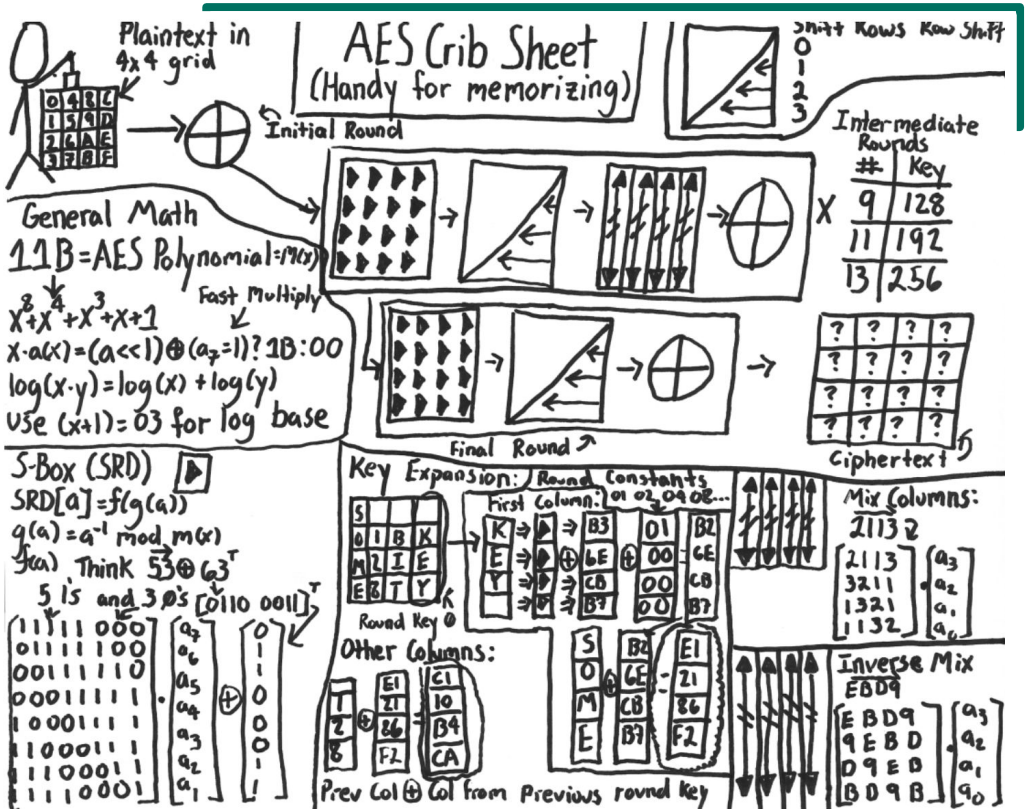
// Why not 2DES? 2DES((k1, k2), m) is insecure

AES - Advanced Encryption Standard

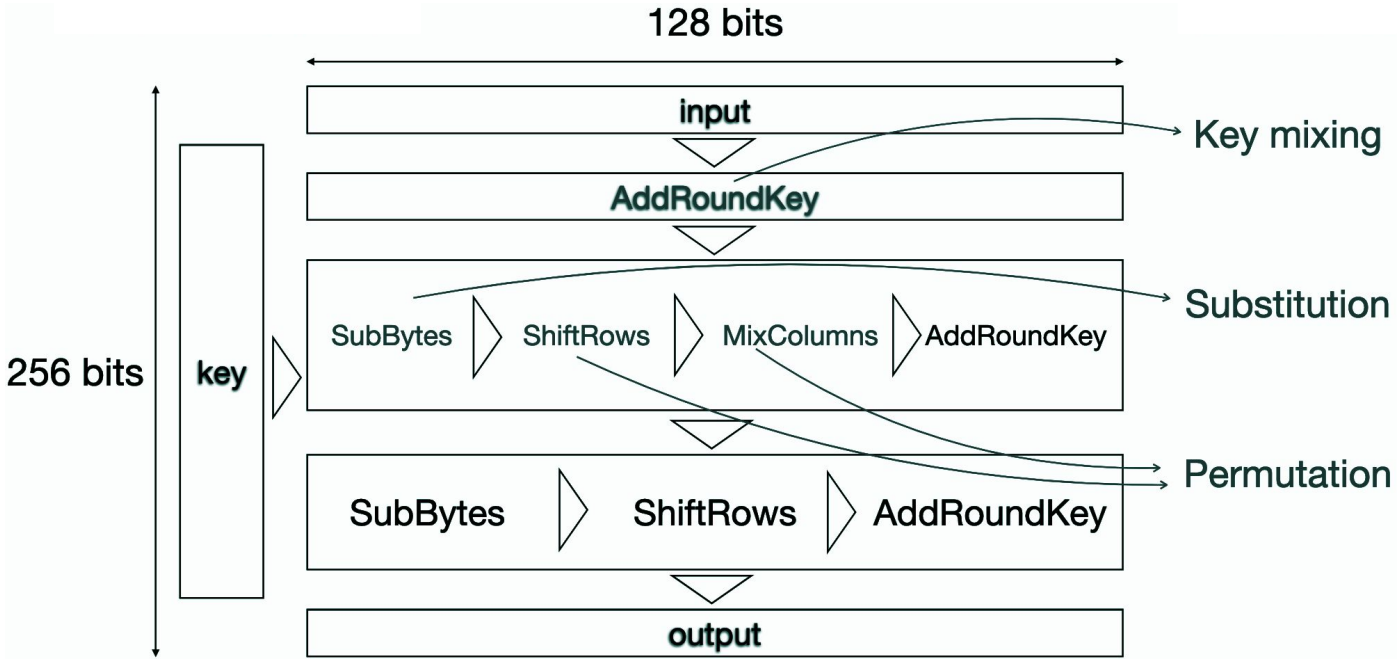
Encryption schema

- Defined in 1998 (as Rijndael), winner of the AES competition and standardized in 2001
- Substitution-Permutation Network
- Block size = 128 bits, variable key size (128, 192, 256 bits)
- Number of rounds is dependent on the key size (10, 12, 14)

AES - Advanced Encryption Standard



AES - Advanced Encryption Standard



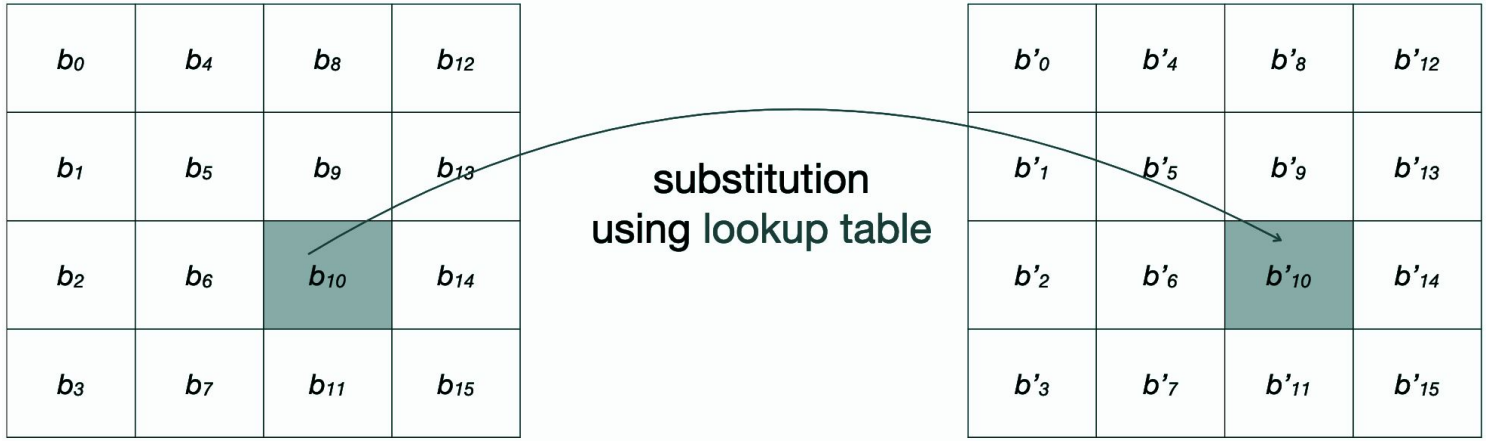
AES - Advanced Encryption Standard

Operations

- **AddRoundKey**: a per-element XOR between the state matrix and the round key matrix
- **SubBytes**: a per-element substitution using a (fixed) table on the state matrix (confusion)
- **ShiftRows**: a cyclic shift of the state matrix rows (diffusion)
- **MixColumns**: matrix multiplication of the state matrix with a fixed matrix (diffusion)

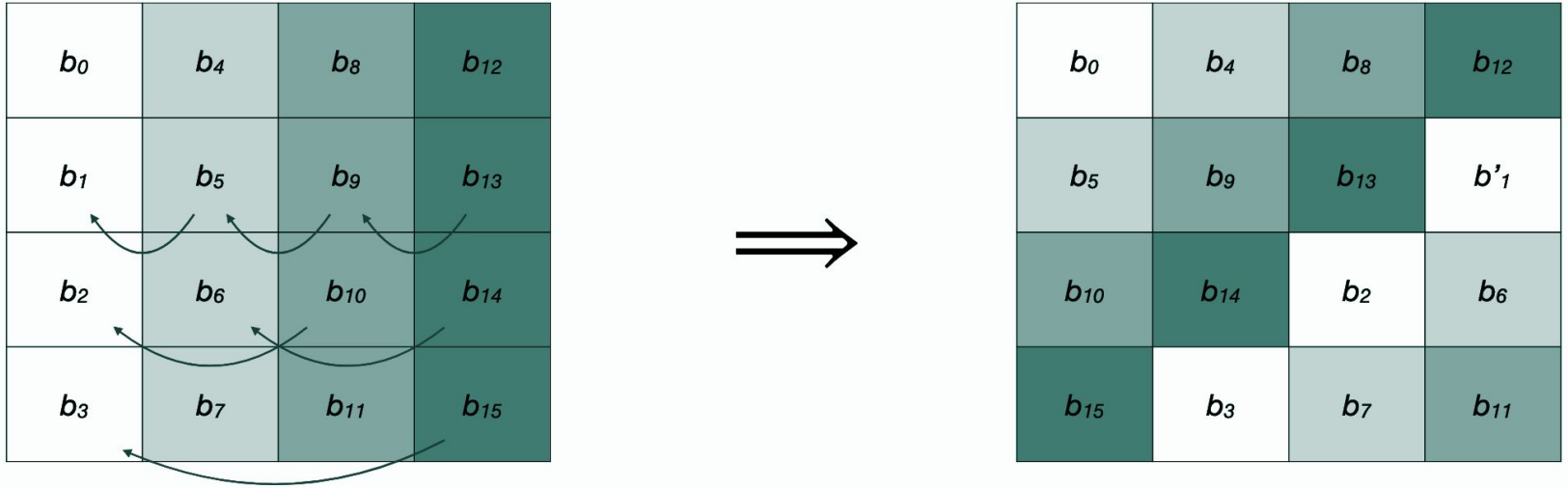
AES - Advanced Encryption Standard

SubBytes



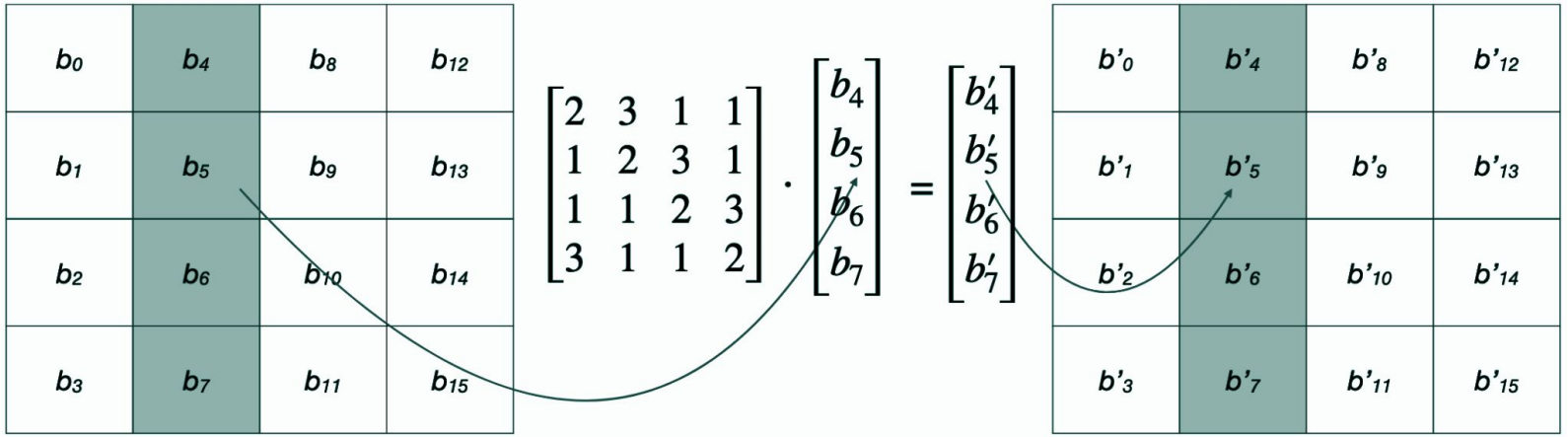
AES - Advanced Encryption Standard

ShiftRows



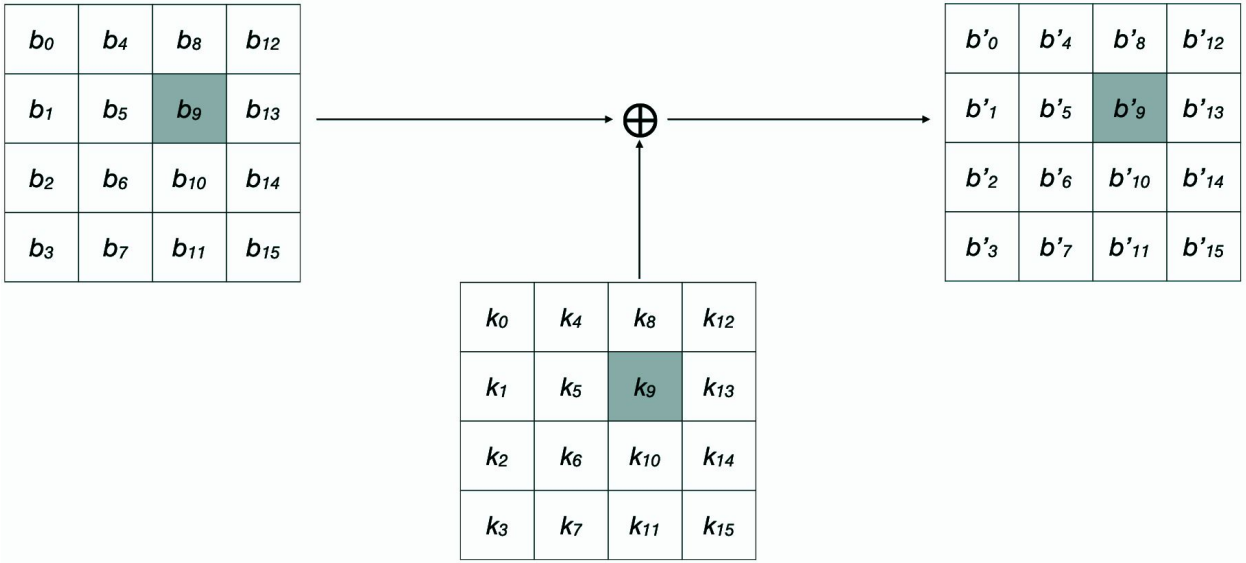
AES - Advanced Encryption Standard

MixColumns



AES - Advanced Encryption Standard

AddRoundKey



AES - Advanced Encryption Standard

Currently, there is no known practical attack

Block Ciphers

AES encrypts fixed-size blocks of data (128 bits), however, real-world data is often larger or smaller than 128 bits

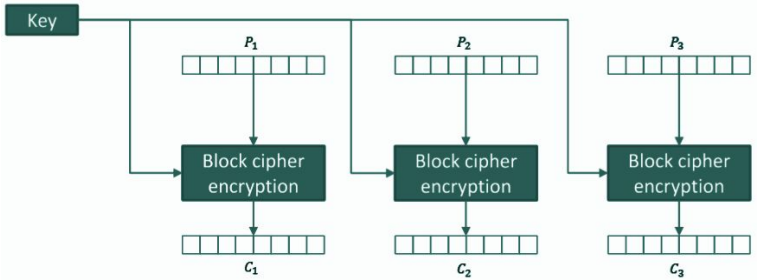
The modes of operation define how to use block ciphers on arbitrary-length messages and are independent of the cipher

- ECB – Electronic Code Book
- CBC – Chipher Block Chaining
- CFB – Chipher Feedback
- OFB – Output Feedback
- CTR – Counter

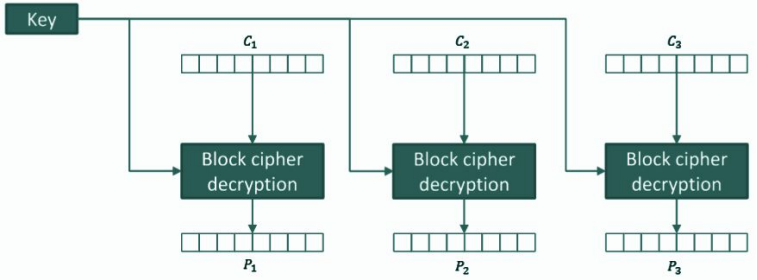
ECB - Electronic CodeBook

We divide the input message into blocks and encrypt each block individually with the same key

Cifratura



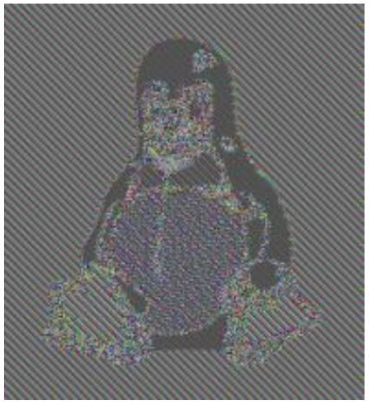
Decifratura



ECB - Electronic CodeBook

Problems

- dividing a message into blocks of the same length is a strong assumption
- equal blocks produce equal ciphertexts
- message structure is preserved



Encryption Oracle

An Encryption Oracle is a service that, given a plaintext message P, returns the corresponding ciphertext C always using the same key

ECB Oracle Attack

Scenario

- an oracle returns c = ECB(k, P || S)
  - P is the plaintext
  - S is a secret string
  - || is the concatenation operator

In this scenario, we can recover S!

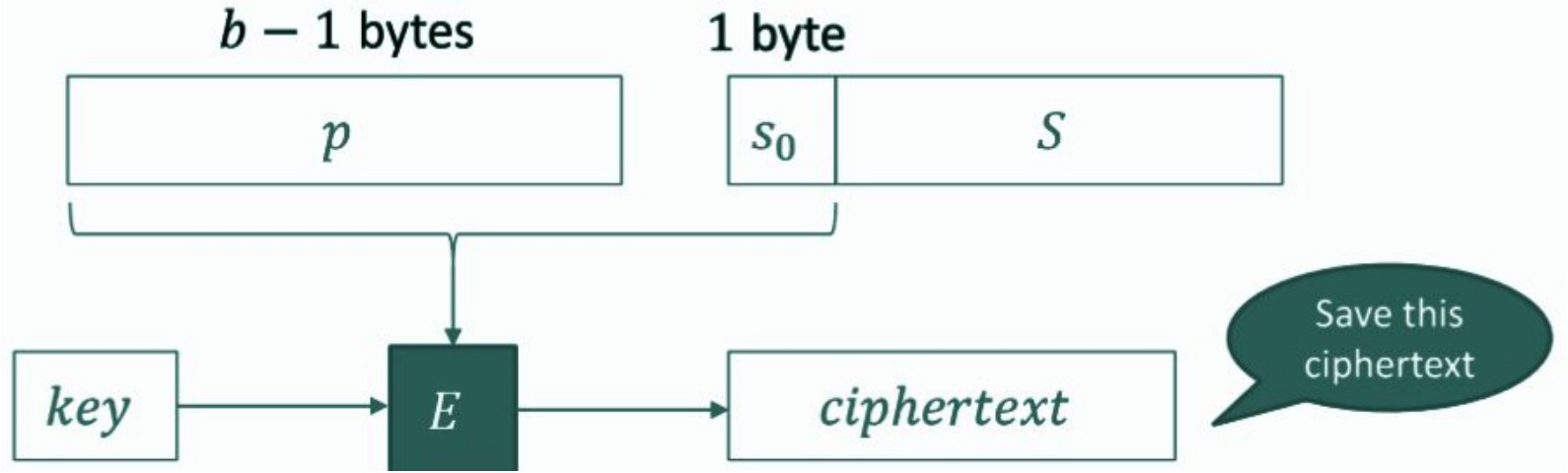
byte-at-a-time Attack

How it work?

- send a message that is 1 byte less than the block and save the result
- brute-force the last byte until we find the corresponding ciphertext
- iterate the procedure one byte at a time

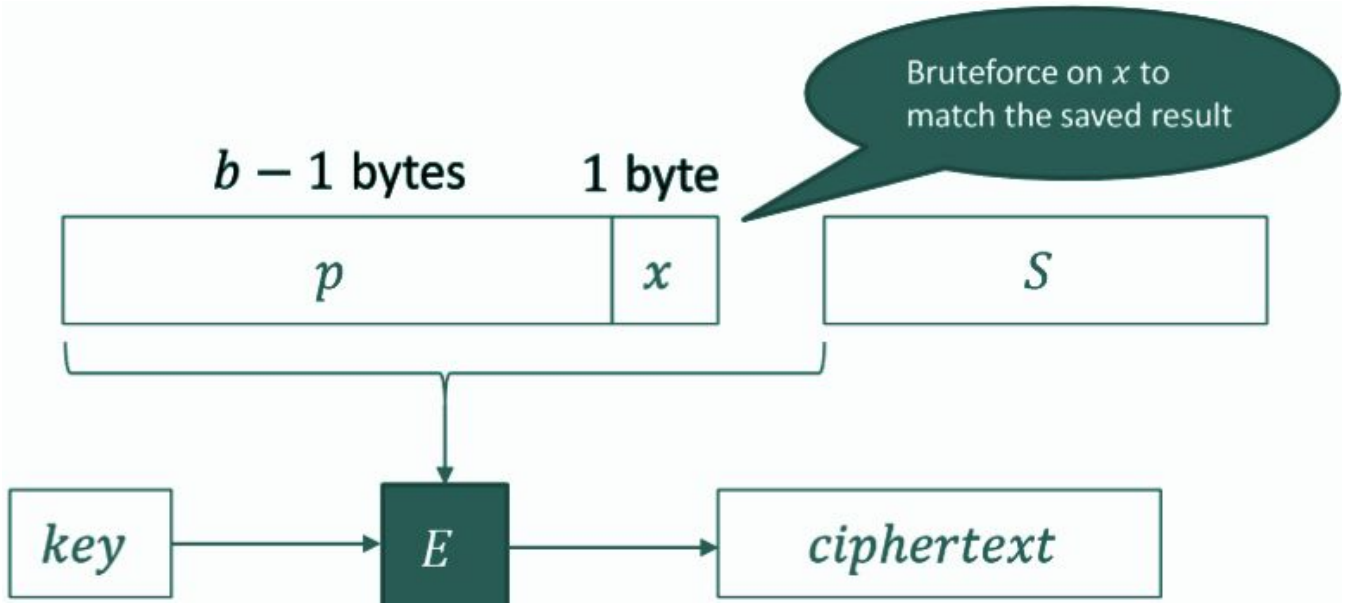
byte-at-a-time Attack

Step 1



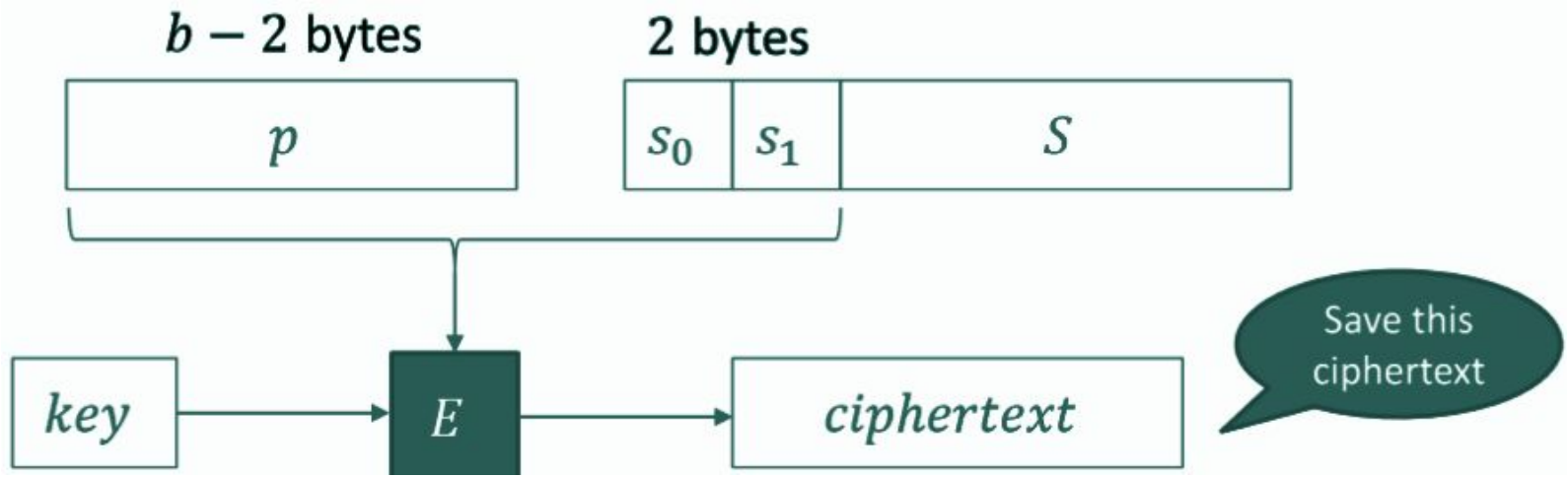
byte-at-a-time Attack

Step 2



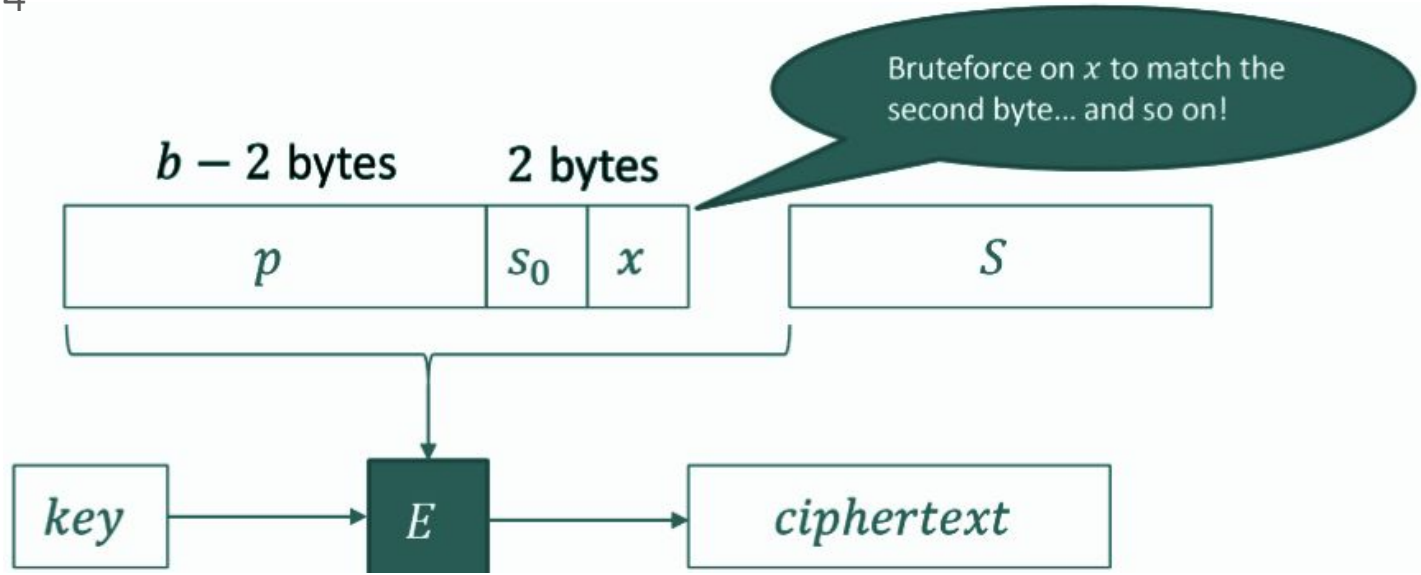
byte-at-a-time Attack

Step 3



byte-at-a-time Attack

Step 4



byte-at-a-time Attack

Split the message into blocks of 16

```
'Agent,\nGreetings' <--- 16  (Block 1)
'. My situation r'  <--- 16  (Block 2)
'eport is as foll'  <--- 16  (Block 3)
'ows:\nAAAAAAAAAAA' <--- 16  (Block 4)
'BBBBBBBBBBBBBBBB'  <--- 16  (Block 5)
'CCCCCCCCCCCCCCCC'  <--- 16  (Block 6)
'\nMy agent identi' <--- 16  (Block 7)
'fying code is:  '  <--- 16  (Block 8)  <---known Block
'picoCTF{ABCDEFG}'  <--- 16  (Block 9)  <---unknown Block
'.Down with the S'  <--- 16  (Block 10) <---known Block
```

Source: https://dr3dd.gitlab.io/cryptography/2018/10/11/Simple-Attack-On-AES-ECB-Mode/

byte-at-a-time Attack

Send the input with less "C"

```
3   'Agent,\nGreetings'    <--- 16  (Block 1)
4   '. My situation r'     <--- 16  (Block 2)
5   'eport is as foll'     <--- 16  (Block 3)
6   'ows:\nAAAAAAAAAAA'    <--- 16  (Block 4)
7   'BBBBBBBBBBBBBBBB'     <--- 16  (Block 5)
8   'CCCCCCCCCCCCCCC\n'    <--- 16  (Block 6)
9   'My agent identif'     <--- 16  (Block 7)
10  'ying code is:  p'     <--- 16  (Block 8) <---known Block
11  'icoCTF{ABCDEFG}.'     <--- 16  (Block 9) <---unknown block
12  'Down with the So'     <--- 16  (Block 10)<---known Block
13
```

Source: https://dr3dd.gitlab.io/cryptography/2018/10/11/Simple-Attack-On-AES-ECB-Mode/

byte-at-a-time Attack

Iterate

```
3   'Agent,\nGreetings'     <--- 16  (Block 1)
4   '. My situation r'      <--- 16  (Block 2)
5   'eport is as foll'      <--- 16  (Block 3)
6   'ows:\nAAAAAAAAAAA'     <--- 16  (Block 4)
7   'ing code is: p%s'      <--- 16  (Block 5)
8   'CCCCCCCCCCCCC\nM'      <--- 16  (Block 6)
9   'y agent identify'      <--- 16  (Block 7)
10  'ing code is:  pi'      <--- 16  (Block 8)    <--- known Block
11  'coCTF{ABCDEFG}.D'      <--- 16  (Block 9)    <--- unknown block
12  'own with the Sov'      <--- 16  (Block 10)   <--- known Block
13
```

Source: https://dr3dd.gitlab.io/cryptography/2018/10/11/Simple-Attack-On-AES-ECB-Mode/

Padding

If the number of bits in the message is not a multiple of the block size, Padding is used. The block is "completed" with bytes with a value equal to the number of missing bytes.

Padding

Idea

- the value of each added byte is the number of added bytes
  - if 3 bytes are missing the padding is $0x03\ 0x03\ 0x03$

Standard

- PKCS#5
- PKCS#7

Padding

Block #0								Block #1							
byte #0	byte #1	byte #2	byte #3	byte #4	byte #5	byte #6	byte #7	byte #0	byte #1	byte #2	byte #3	byte #4	byte #5	byte #6	byte #7
'S'	'U'	'P'	'E'	'R'	'S'	'E'	'C'	'R'	'E'	'T'	'1'	'2'	'3'	0x02	0x02
'S'	'U'	'P'	'E'	'R'	'S'	'E'	'C'	'R'	'E'	'T'	'1'	'2'	0x03	0x03	0x03
'S'	'U'	'P'	'E'	'R'	'S'	'E'	'C'	'R'	'E'	'T'	0x05	0x05	0x05	0x05	0x05
'S'	'U'	'P'	'E'	'R'	'S'	'E'	'C'	0x08	0x08	0x08	0x08	0x08	0x08	0x08	0x08

Source: https://pentesterlab.com/exercises/padding_oracle

ECB - Electronic CodeBook

Problems

- dividing a message into blocks of the same length is a strong assumption
- equal blocks produce equal ciphertexts
- message structure is preserved

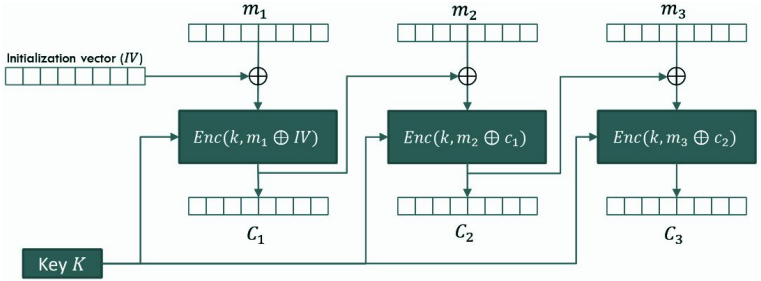
Idea

- use the previously encrypted block to "destroy" the message structure

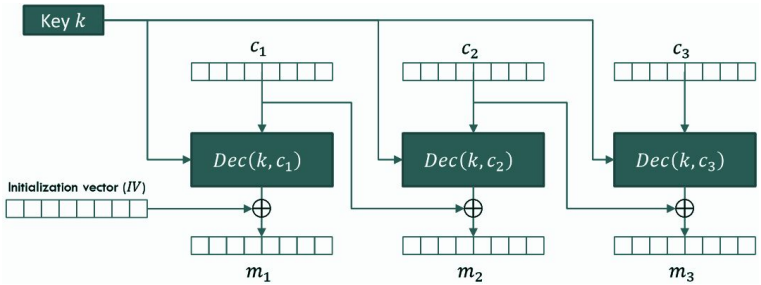
CBC - Cipher Block Chaining

We use an additional random number, called Initialization Vector (IV) to introduce randomness into the first block

Encryption



Decryption



CBC - Cipher Block Chaining

Plaintext structure is no longer maintained

- same plaintext block repeated results in different cipher blocks
- ECB Oracle attack does not work due to IV

//

- IV is not a key, it should be different for each cipher

CBC - Cipher Block Chaining

Problems

- encryption cannot be in parallel
- a transmission error in the ciphertext leads to an incorrect decryption on two blocks
- attacks are often due to implementation errors

IV as key

Scenario

- a server implements a CBC scheme using the (fixed) key as IV (without revealing it)

You can ask the server to decrypt a message. Is it possible to recover the key?

IV as key

Strategy

- send a message to the server with 2 equal blocks BB
- get
  - $P_1 = Dec(k, B) \oplus IV$ e $P_2 = Dec(k, B) \oplus B$
- compute
  - $P_1 \oplus P_2 \oplus B = IV = k$

Padding Oracle Attack

Scenario

- We have a target ciphertext (with padding) to decrypt
- We have a "padding oracle"
  - a server that, given a ciphertext, simply says whether the padding is correct or not

Padding Oracle Attack

Attack (for 1 block ciphertext C)

- create a random block R
- append the target block obtaining $R||C$
- discover the length of the padding using the oracle
- decrypt one byte at a time using the oracle

Padding Oracle Attack

Step 1: look for a "correct padding" message

- Try to decrypt $R||C$
  - With high probability, you will get "wrong padding"
- Keep changing the last byte of R in order to get "correct padding"

Now you know that the decryption of $R||C$ ends in $0x01$ or $0x02$ $0x02$ or $0x03$ $0x03$ $0x03$ or ..

Padding Oracle Attack

Step 2: find the length of the padding

- let R now be the block that gives "correct padding"

Change randomly

- the first byte of R
  - if it still gives correct padding, the padding length is $b - 1$ or less
- the second byte of R
  - if it still gives correct padding, the padding length is $b - 2$ or less, and so on

If you reach an "incorrect padding" on the kth byte, you found the padding length!

Padding Oracle Attack

Step 3: decrypt the padding bytes

- we discovered n bytes, where n is the padding length
- In order to get them, just XOR the corresponding bytes of R with the padding bytes

Padding Oracle Attack

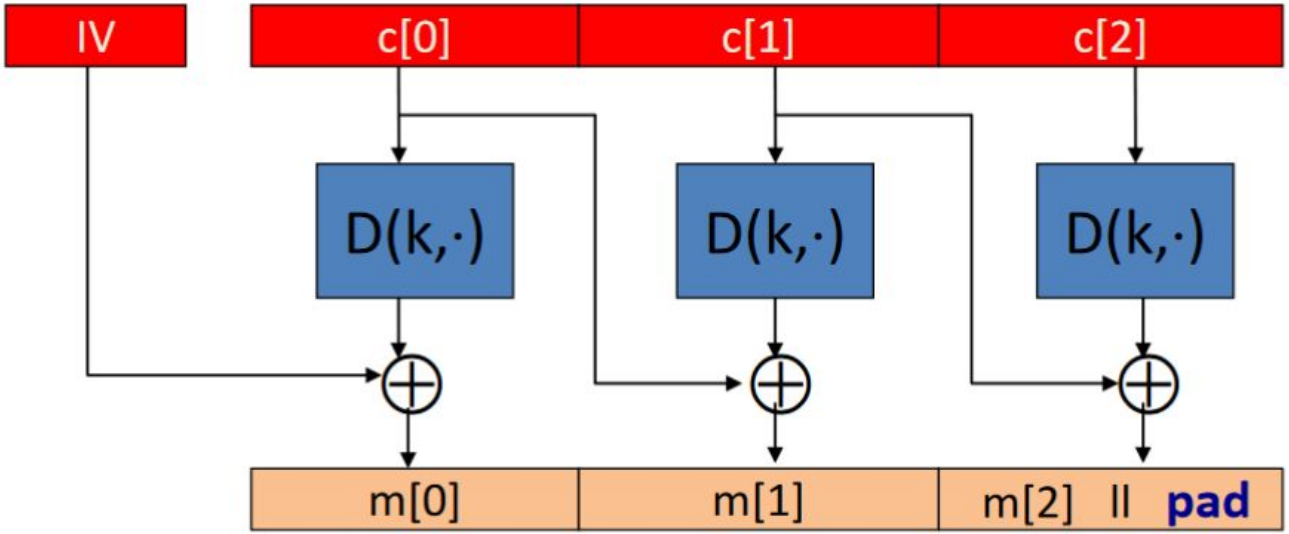
Step 4: decrypt subsequent bytes

- To get one more byte, we need to "increase the padding"
  - XOR the padding bytes with $n \oplus (n + 1)$ (this just increase them by 1)

Repeat from step 1 using the first non-padding byte instead of the last one

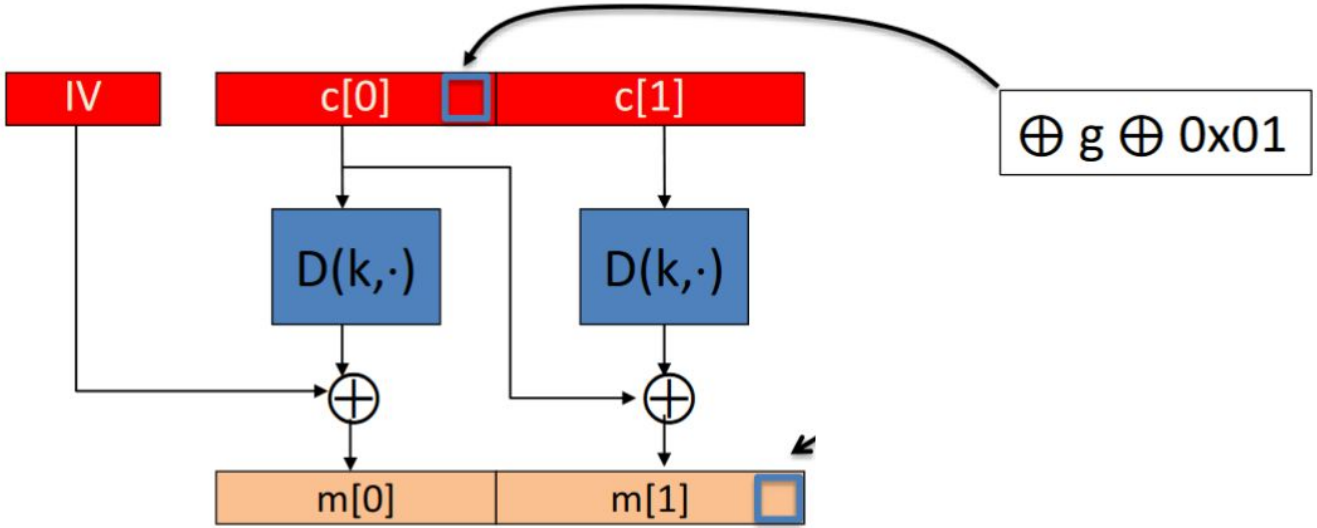
Padding Oracle Attack

An attacker knows c[0], c[1], c[2] and wants m[1]



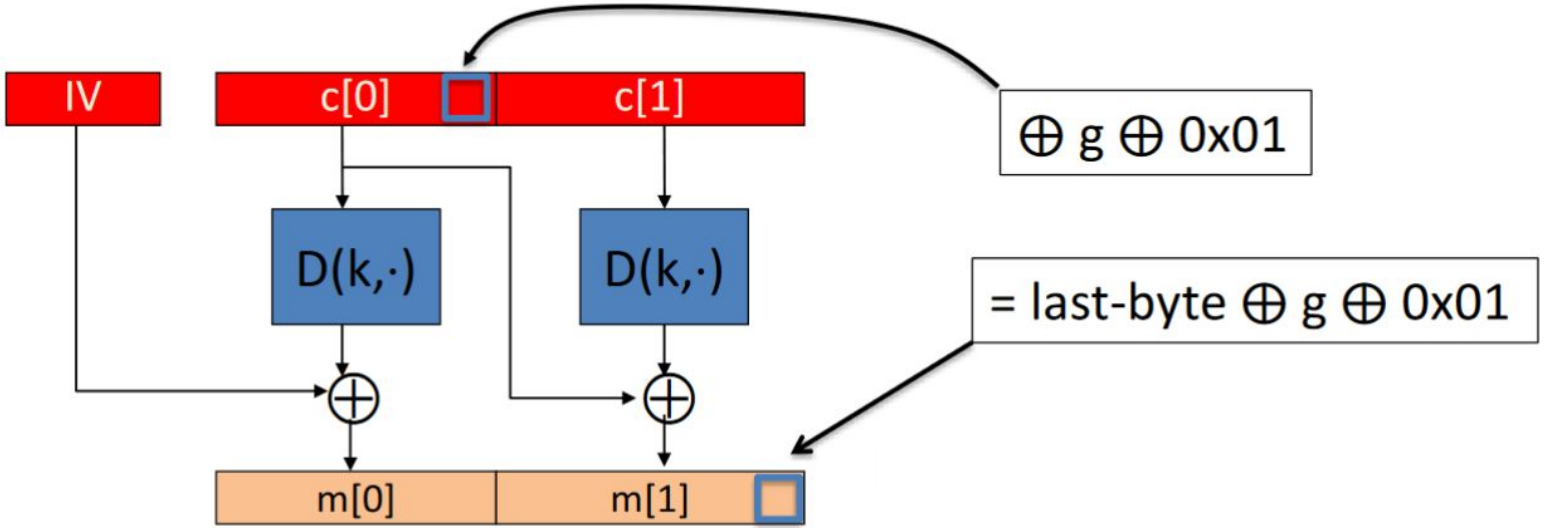
Source: https://crypto.stanford.edu/~dabo/courses/OnlineCrypto/

Padding Oracle Attack



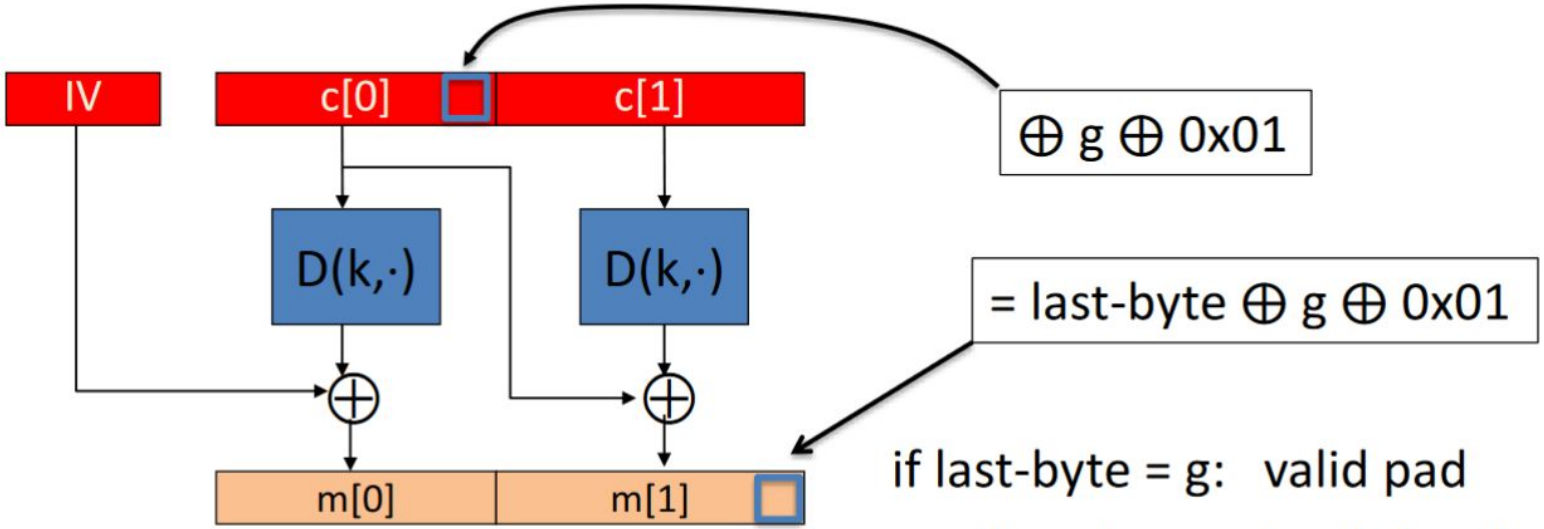
Source: https://crypto.stanford.edu/~dabo/courses/OnlineCrypto/

Padding Oracle Attack



Source: https://crypto.stanford.edu/~dabo/courses/OnlineCrypto/

Padding Oracle Attack



Source: https://crypto.stanford.edu/~dabo/courses/OnlineCrypto/

Bitflipping Attack

An attacker can modify the ciphertext in such a way that it changes the plaintext in a predictable way. The attacker is unable to obtain the plaintext.

Scenario

- We have a partially controlled CBC-encrypted message, with some secret information inside
- it is possible to "sacrifice" a piece of plaintext in order to edit the secret part

Bitflipping Attack

Attack outline

- reserve an entire block with our controlled data and XOR it with its plaintext value and the value that we want to put in the secret part
- paying the price of destroying our controlled part, we control the secret without controlling the key

Bitflipping Attack

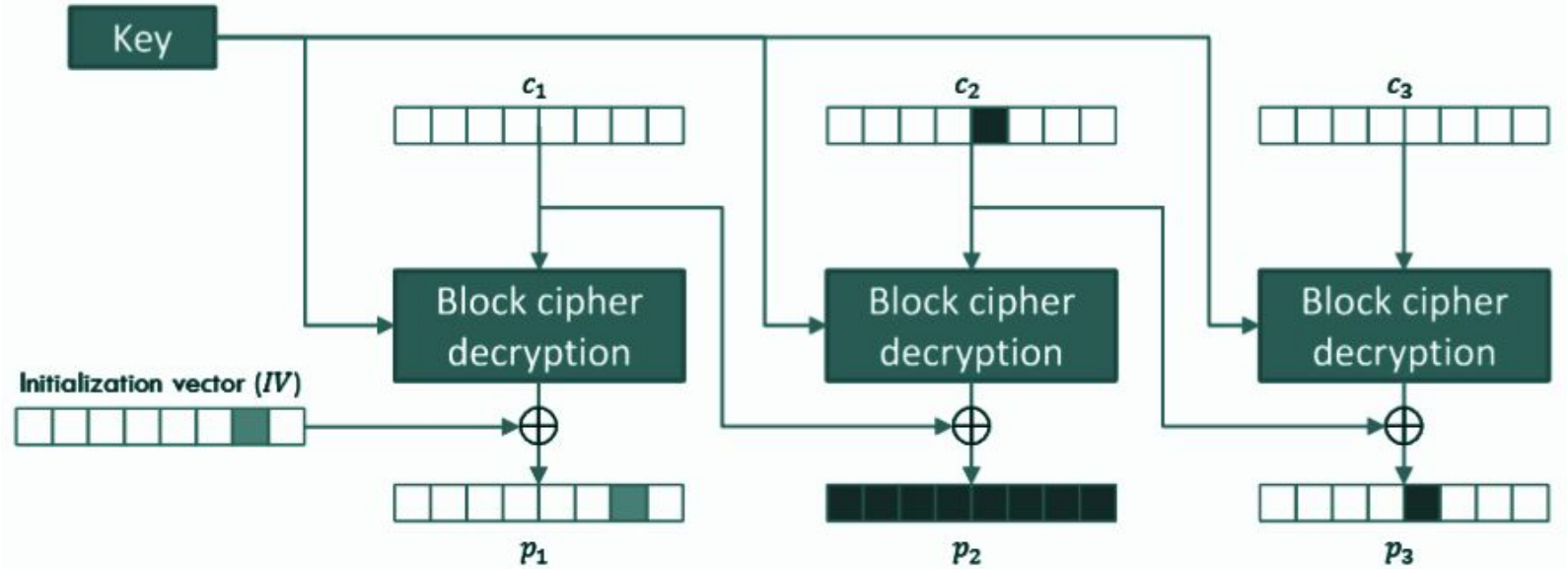
The decryption process in CBC mode is performed as

- $P_1 = Dec(k, C_1) \oplus IV$
- $P_i = Dec(k, C_i) \oplus C_{i-1}$ // with $1<i\leq nb$, where nb is the number of blocks

Knowing the position of the target byte, it is possible to modify the position of the corresponding ciphertext in the previous ciphertext block.

Example: If you change one byte in the ciphertext C_{i-1}, then P_i will be changed since C_{i-1} affects the plaintext P_i

Bitflipping Attack



Bitflipping Attack

Black: we modify a byte of C_2. This affects the corresponding byte in the next plaintext block P_3 and the corresponding complete plaintext block P_2 that has the same index as the modified ciphertext. This can be seen as an error.

Green: we modify a byte IV. This affects only the corresponding byte in the first plaintext P_1. If the target plaintext is in the first block, it will not leave a trace.

CBC - Cipher Block Chaining

Problems

- Encryption cannot be in parallel
- A transmission error in the ciphertext leads to an incorrect decryption on two blocks
- Attacks are often due to implementation errors

Idea

- Use a counter

CTR - Counter Mode

We do not use the block cipher as a cipher, but as something that generates a stream to be used as input for one-time pads

- we use an additional random number *N* called **nonce** (number used once) to ensure that the stream is unique

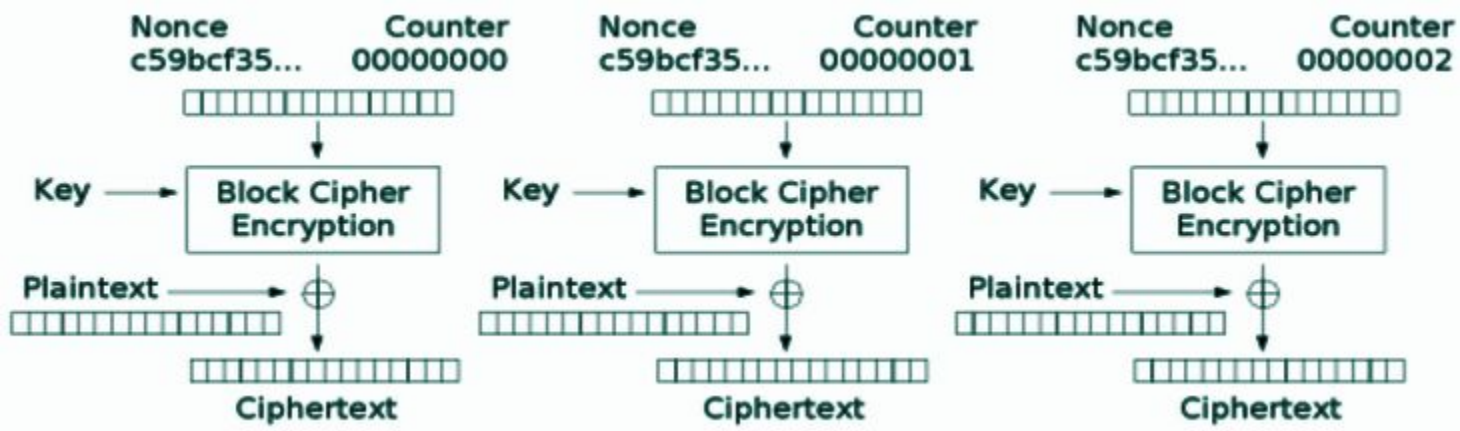
- we encrypt strings formed by the nonce concatenated to a counter with the block cipher (and a key k) to generate some bytes

CTR - Counter Mode

Example

- N = 12345678
- encrypt 1234567800000000 to generate the first 16 bytes
- encrypt 1234567800000001 to generate another 16 bytes
- encrypt 1234567800000002 and so on, until the desired number of bytes is reached

CTR - Counter Mode



Challenge

Crypto 07 - PyCryptutorial 1

Challenges

CR_1.04 - Secure Padding
CR_1.07 - See You In The Center
CR_1.08 - Redacted
CR_1.10 - notadmin
CR_1.11 - Padding Oracle
CR_1.12 - Predictable IV
CR_1.13 - DesOracle